



Universidad
Tecnológica
del Perú

Universidad Tecnológica del Perú

Guía De Laboratorio 06

Sistema Inteligente de Apoyo Pedagógico y Gestión del Aprendizaje para Instituciones Educativas

INTEGRANTES – GRUPO 6:

Rivera Castañeda, Jesús Martín	- U21213646
Quispe Fernández, Alonso	- U20208103
Alanya Guevara, Ben Andre	- U21315933
Ninatanta Fernandez, Renato Ventura	- U21205210

ASIGNATURA:

Curso Integrador II: Software

DOCENTE:

Barturen Huaman, Renzo Renato

SECCIÓN:

31738

Lima – Perú

ÍNDICE

1. Resultado y alcance del incremento	3
1.1 Producto mínimo de la semana	3
2. Correspondencia con el sílabo	3
3. Arquitectura del incremento	4
3.1 Responsabilidades por componente	4
3.2 Flujo de la consulta	4
3.3 Estructura de carpetas	5
4. Modelo físico y reglas de integridad	5
4.1 Tablas del incremento	5
4.2 Reglas de integridad	5
4.3 Índices basados en consultas	6
5. Migraciones y datos sintéticos	6
6. Repository, Service y proyección de datos	7
7. Tiempo y zona horaria	7
8. Calidad de datos	7
9. Preparación del entorno	8
10. API implementada	8
10.1 Catálogo de cursos	8
10.2 Contenidos disponibles por curso y fecha	8
10.3 Manejo de errores	9
11. Integración con el Front-End (React)	9
12. Comprobación integral y evidencias	9
13. Riesgos conectados al incremento	10
14. Entregables y estado de evidencia (E0–E11)	10
15. Checklist de cierre	11

1. Resultado y alcance del incremento

Este avance corresponde a la Semana 6 del Curso Integrador II: Software y da continuidad al backlog, prototipo, repositorio, incremento, registro de riesgos y catálogo de KPI/SLI elaborados en las guías de laboratorio anteriores sobre ESTUD-IA, el Sistema Inteligente de Apoyo Pedagógico y Gestión del Aprendizaje. El objetivo de esta etapa es construir el primer flujo vertical del Back-End: persistir el catálogo académico en una base de datos relacional, publicar un primer conjunto de consultas REST y conectar el Front-End para que deje de simular los datos que hasta ahora mostraba con información estática.

1.1 Producto mínimo de la semana

- Base de datos MySQL reproducible en el entorno local del equipo (servicio local o contenedor con imagen mysql:8).
- Migraciones Laravel para el catálogo de cursos, temas, kpis y usuarios tipos de contenido, módulos y contenidos.
- Seeder con datos sintéticos equivalentes al fixture que el Front-End usa desde la Semana 5.
- API GET del contenido mostrado en el frontend para los roles: estudiante y administrador.
- Capas Controller – Service – Repository sobre Eloquent, con Resources que no exponen los modelos directamente.
- Buscador en React que obtiene catálogo y contenidos mediante Axios, reemplazando el fixture estático.
- Pruebas unitarias del Front-End y pruebas de Back-End con PHPUnit, incluyendo casos negativos de integridad.

2. Correspondencia con el sílabo

La siguiente tabla adapta la correspondencia planteada por la guía del docente a las tecnologías reales del proyecto.

Tema de la semana 6	Aplicación en ESTUD-IA	Evidencia
Diseño e implementación de base de datos	Modelo físico de la DB, tipo de contenido, módulo y contenido, con claves, restricciones e índices en MySQL.	Migraciones Laravel y consulta del catálogo.
Replicación	Revisión del binary log de MySQL y verificación de que el servicio local opera como instancia única, sin réplicas conectadas.	SHOW VARIABLES LIKE 'log_bin'; SHOW REPLICAS;
Administración de base de datos	Servicio local con variables de entorno en .env y revisión del historial de migraciones.	php artisan migrate:status

Tema de la semana 6	Aplicación en ESTUD-IA	Evidencia
Patrones de acceso a datos	Rutas api, Controller, Service con stored procedure, proyección mediante API Resource.	Clases CursoController, ContenidoController, CursoResource, ContenidoResource.
Calidad de datos	Reglas NOT NULL, UNIQUE, CHECK/validaciones de Form Request y FOREIGN KEY verificadas con casos negativos.	Pruebas PHPUnit y tabla de casos negativos.

3. Arquitectura del incremento

3.1 Responsabilidades por componente

Componente	Responsabilidad	No debe hacer
React	Capturar filtros (curso y fecha), representar estados de la búsqueda y consumir JSON.	Construir consultas SQL o ignorar los estados de error.
Cliente Axios	Centralizar la URL base y los parámetros de cada petición HTTP.	Repetir lógica de peticiones dentro de los componentes.
Controller (Laravel)	Traducir la petición HTTP, validar parámetros con un Form Request y delegar al Service.	Consultar la base de datos directamente.
Service	Contiene la función que obtendrá las consultas mediante stored procedure.	Formatear HTML o conocer detalles del navegador.
Repository (Eloquent - Opcional)	Encapsular la consulta persistente y devolver una proyección mediante Resource.	Tomar decisiones de presentación.
MySQL	Garantizar identidad, referencias, valores válidos y unicidad.	Resolver reglas que dependen de servicios externos.

3.2 Flujo de la consulta

1. React envía GET /api/v1/contenidos con cursoCodigo mediante Axios.
2. El Controller valida el formato de ambos parámetros con un Form Request.
3. El Service es el enlace con la bd, donde se obtendrá la consulta realizada.

4. El Repository ejecuta la consulta sobre Eloquent y aplica la proyección a un Resource. (Opcional)
5. MySQL filtra e integra las tablas curso, módulo, tipo_contenido y contenido.

3.3 Estructura de carpetas

```

estud-ia/
├── frontend/           (React + Vite)
│   ├── src/features/contenidos/
│   │   ├── api/contenidosApi.js
│   │   ├── hooks/useBuscadorContenidos.js
│   │   └── components/
│   └── backend/       (Laravel)
│       ├── app/Http/Controllers/Api/CursoController.php
│       ├── app/Http/Controllers/Api/ContenidoController.php
│       ├── app/Http/Requests/BuscarContenidosRequest.php
│       ├── app/Http/Resources/CursoResource.php
│       ├── app/Http/Resources/ContenidoResource.php
│       ├── app/Services/ContenidoDisponibilidadService.php
│       ├── app/Repositories/ContenidoRepository.php
│       ├── app/Models/{Curso,Modulo,TipoContenido,Contenido}.php
│       ├── database/migrations/
│       └── database/seeders/

```

4. Modelo físico y reglas de integridad

4.1 Tablas del incremento

- curso: catálogo de cursos (equivalente a specialty). Código único, nombre y estado activo.
- tipo_contenido: catálogo independiente de tipos de contenido (equivalente a clinic_location): Video, Lectura, Ejercicio, Quiz.
- modulo: temas de un curso (equivalente a professional). Cada módulo pertenece a un único curso.
- contenido: unidad publicable (equivalente a appointment_slot). Pertenece a un módulo y a un tipo de contenido, con fecha/hora de publicación y estado.

4.2 Reglas de integridad

Regla del caso	Mecanismo	Razón
Código de curso irrepitable	UNIQUE en curso.codigo	Evita dos identidades para el mismo curso.
Módulo asociado a un curso existente	FOREIGN KEY curso_id, ON DELETE RESTRICT	Conserva la integridad referencial del catálogo.

Regla del caso	Mecanismo	Razón
Contenido asociado a un módulo y a un tipo existentes	FOREIGN KEY modulo_id y tipo_contenido_id, ON DELETE RESTRICT	Impide contenidos huérfanos o de tipo inexistente.
Un módulo sin dos contenidos publicados a la misma hora	UNIQUE (modulo_id, disponible_desde)	Impide una colisión exacta de publicación.
Estado dentro del vocabulario permitido	CHECK estado IN ('BORRADOR','PUBLICADO','ARCHIVADO')	Detiene valores que la aplicación no comprende.
Nombre y código no vacíos	NOT NULL + validación de Form Request (regla required y regex)	Evita registros sin identidad legible.

MySQL 8 admite restricciones CHECK desde la versión 8.0.16, por lo que estas reglas se declaran en la propia migración, igual que en PostgreSQL; la diferencia frente a la guía es de sintaxis, no de criterio.

4.3 Índices basados en consultas

La búsqueda de contenidos filtra por estado y rango de fecha, y se relaciona con el módulo; por ello se crea el índice idx_contenido_busqueda sobre (estado, disponible_desde, modulo_id), equivalente a ix_slot_availability_search de la guía. Las claves primarias y las restricciones UNIQUE ya generan sus propios índices; MySQL, igual que PostgreSQL, no crea automáticamente un índice sobre una columna de clave foránea salvo que se declare explícitamente.

5. Migraciones y datos sintéticos

Laravel ordena y versiona las migraciones mediante el prefijo de fecha y hora del nombre de archivo, de forma análoga a como Flyway ordena V1, V2 y V3 por su número de versión. Una migración ya fusionada a main se trata como inmutable: si el modelo cambia, se crea una nueva migración en lugar de editar una existente, igual que la guía indica para Flyway.

- 2026_XX_XX_000001_create_curso_and_tipo_contenido_table — catálogo independiente (equivalente a V1).
- 2026_XX_XX_000002_create_modulo_and_contenido_table — módulos, contenidos, restricciones e índice de búsqueda (equivalente a V2).
- database/seeder/ContenidoDemoSeeder.php — datos sintéticos ejecutados con php artisan db:seed (equivalente funcional a V3).

php artisan migrate:status permite verificar, de forma análoga a una consulta sobre flyway_schema_history, qué migraciones están aplicadas en el entorno local de cada integrante.

6. Repository, Service y proyección de datos

Los modelos Eloquent definen los atributos \$fillable, casts y relaciones necesarias. No se exponen directamente en las respuestas HTTP. El Controller recibe la petición y delega la operación al Service, que contiene la lógica de negocio y ejecuta el Stored Procedure mediante DB::select('CALL nombre_sp(?)', [...]).

El Stored Procedure se encarga de las consultas SQL y respeta las relaciones existentes. El Service valida los resultados y devuelve los datos necesarios al Controller, sin utilizar Resources ni construir respuestas HTTP. El Controller maneja la respuesta y utiliza el Resource para transformar los datos.

El Resource transforma los datos antes de enviarlos al frontend y define exactamente qué campos serán públicos. De esta manera, no se expone directamente todo el modelo Eloquent ni información interna como claves foráneas o timestamps que no sean necesarios.

7. Tiempo y zona horaria

La base de datos utiliza MySQL y las fechas se almacenan según el tipo definido para cada campo. Los campos created_at y updated_at corresponden a los timestamps de auditoría y no deben utilizarse para representar fechas propias de la lógica de negocio.

Cuando una funcionalidad recibe una fecha local, el Service utiliza America/Lima para calcular el intervalo correspondiente desde el inicio de ese día hasta el inicio del día siguiente. La consulta utiliza el inicio de forma inclusiva y el final de forma exclusiva ($\geq$ inicio y $<$ fin), evitando perder registros y manteniendo una regla clara para el manejo de fechas.

8. Calidad de datos

Dimensión	Pregunta	Control de esta práctica
Exactitud	¿Representa un contenido válido?	Estado permitido y asociación a un módulo y tipo existentes.
Compleitud	¿Falta un dato obligatorio?	NOT NULL en identidad, relaciones y fecha de publicación.
Consistencia	¿Las referencias y vocabularios coinciden?	FOREIGN KEY y CHECK sobre estado.
Unicidad	¿Existe un duplicado?	Código de curso único y combinación módulo/fecha de publicación.

Dimensión	Pregunta	Control de esta práctica
Actualidad	¿El dato sigue vigente?	activo y estado como parte del filtro de la consulta.

9. Preparación del entorno

Se trabaja en el mismo repositorio de las semanas anteriores, agregando la carpeta backend al lado de frontend. La referencia usa PHP 8.3 o superior, Composer, Node 20 o superior, npm y MySQL 8, con un servicio local (XAMPP/Laragon) o un contenedor Docker equivalente al que la guía usa para MySQL.

```
git switch -c feature/semana6-api-cursos
php --version
composer --version
node --version
npm --version
mysql --version
```

Igual que en la guía, .env permanece fuera del repositorio y contiene las credenciales locales de la base de datos (DB_DATABASE, DB_USERNAME, DB_PASSWORD); las variables se cargan automáticamente por Laravel y no deben imprimirse en la terminal ni en las capturas de evidencia.

10. API implementada

10.1 Catálogo de cursos

GET /api/v1/cursos devuelve los cursos activos ordenados por nombre:

```
[
  {"codigo":"MAT101","nombre":"Matemática Básica"},
  {"codigo":"COM101","nombre":"Comprensión Lectora"}
]
```

10.2 Contenidos disponibles por curso y fecha

GET /api/v1/contenidos?cursoCodigo=MAT101&fecha=2026-09-03 devuelve los contenidos publicados de ese curso para esa fecha, en hora de Lima:

```
[
  {"id":"MAT101-20260903-A","curso":"Matemática Básica",
    "modulo":"Álgebra Básica","tipo":"Ejercicio",
    "fecha":"2026-09-03","hora":"09:00","estado":"PUBLICADO"},
  {"id":"MAT101-20260903-B","curso":"Matemática Básica",
    "modulo":"Álgebra Básica","tipo":"Video",
    "fecha":"2026-09-03","hora":"11:30","estado":"PUBLICADO"}
]
```

Una fecha sin contenidos publicados devuelve HTTP 200 con un arreglo vacío, igual que la guía distingue entre 'sin resultados' y un error.

10.3 Manejo de errores

En ESTUD-IA, el Request valida los datos enviados por el frontend antes de llegar al Service. Si la información no cumple las reglas establecidas, Laravel devuelve HTTP 422 con los campos que presentan errores. El Service controla los casos en los que no existen datos o se produce algún problema durante la operación, mientras que el Controller se encarga de devolver el código HTTP y el mensaje correspondiente. De esta forma, cada capa mantiene su responsabilidad y no se exponen consultas SQL, trazas ni información interna del sistema.

11. Integración con el Front-End (React)

El frontend de ESTUD-IA utiliza React para consumir los endpoints del backend. Los archivos API, como contenidosApi.js, centralizan las peticiones mediante Axios y envían los parámetros necesarios para cada operación. Los hooks administran la comunicación con el backend y los estados de la interfaz, como idle, loading, success, empty y error, permitiendo mostrar al usuario si una consulta está cargando, obtuvo resultados, no encontró información o presentó un error.

Cuando el usuario realiza una nueva búsqueda, se cancela la petición anterior mediante AbortController. Esto evita que una respuesta anterior sobrescriba los resultados de una búsqueda más reciente. Si el backend no responde correctamente, ESTUD-IA conserva los datos ingresados por el usuario y muestra el estado de error correspondiente, sin generar información que no haya sido proporcionada por la API.

12. Comprobación integral y evidencias

Caso	Acción	Resultado esperado
Catálogo	GET /api/v1/cursos	Cursos activos ordenados.
Matemática	MAT101 y 03/09/2026	Dos contenidos en hora de Lima.
Comprensión Lectora	COM101 y 03/09/2026	Un contenido publicado.
Sin resultados	MAT101 y 04/09/2026	HTTP 200 y arreglo vacío.
Código inválido	mat#101	HTTP 422 con detalle de validación.
API detenida	Buscar desde React	Estado de error y filtros conservados.
Restricción	Insertar contenido con módulo inexistente	MySQL rechaza la fila.
Migraciones	php artisan migrate:status	Todas las migraciones de la semana aplicadas.

13. Riesgos conectados al incremento

Se retoman los riesgos R-01 a R-11 registrados en las guías anteriores y se agrega un riesgo propio de esta semana, originado en la diferencia entre el motor de referencia de la guía (PostgreSQL) y el motor real del proyecto (MySQL).

ID	Riesgo formulado	Disparador	Control / respuesta	Indicador relacionado
R-12	Una regla escrita para PostgreSQL (tipo, sintaxis de CHECK o función) podría no tener un equivalente directo en MySQL y quedar sin aplicarse.	Migración que falla o restricción que se ejecuta sin efecto en MySQL.	Probar cada migración en el motor real antes de fusionar y documentar la equivalencia elegida.	DEL-01

14. Entregables y estado de evidencia (E0–E11)

ID	Entregable	Contenido mínimo	Estado
E0	Inicio controlado	Issue, rama feature/semana6-api-cursos, commit base.	Pendiente
E1	Modelo físico y migraciones	Tablas curso, tipo_contenido, módulo y contenido con integridad referencial.	Completado (propuesta)
E2	Datos sintéticos	Seeder equivalente al fixture usado por React desde la Semana 5.	Pendiente
E3	API de catálogo	GET /api/v1/cursos.	Pendiente
E4	API de disponibilidad	GET /api/v1/contenidos por curso y fecha.	Pendiente
E5	Pruebas negativas de integridad	Casos que validan cada restricción de la Sección 4.2.	Pendiente
E6	Pruebas automatizadas del Back-End	Suite PHPUnit, incluyendo un caso contra MySQL real.	Pendiente
E7	Integración del Front-End	React consumiendo la API mediante Axios, sin datos fijos.	Pendiente
E8	Pruebas del Front-End	Suite de pruebas del cliente HTTP y del buscador.	Pendiente
E9	Control técnico	Salida de pruebas, build y consola sin errores.	Pendiente

ID	Entregable	Contenido mínimo	Estado
E10	Comparación y evidencia	Capturas, JSON de respuesta y explicación de decisiones.	Pendiente
E11	Defensa	Presentación y Pull Request con revisión por pares.	Pendiente

15. Checklist de cierre

- El modelo físico y sus restricciones están delimitados y justificados.
- Cada migración sigue la convención de Laravel y no se edita una vez fusionada.
- La proyección de datos (Resource) no expone el modelo Eloquent completo.
- La zona horaria se calcula en el Service, con inicio inclusivo y fin exclusivo.
- La API responde con Problem-Details equivalente ante parámetros inválidos.
- El Front-End en React reemplaza el fixture estático y conserva los cinco estados de búsqueda.
- Los riesgos, el backlog y el Pull Request se actualizarán con la nueva evidencia.